

Detailed React Interview Questions

1. What is React?

React is a **JavaScript library used to build user interfaces**, especially for web applications.

Instead of reloading the entire page, React updates only the required parts using something called the **Virtual DOM**, making apps faster and smoother.

 Example:

Web apps like dashboards, forms, and social media feeds.

2. What are Components?

Components are **reusable building blocks** of a React application.

Each component represents a part of UI (like navbar, button, form).

 Types:

- Functional (modern, recommended)
- Class (old)

 Example:

A “UserCard” component can be reused multiple times.

3. What is JSX?

JSX is a syntax that allows you to write **HTML inside JavaScript**.

React uses JSX to describe what the UI should look like.

```
const element = <h1>Hello</h1>;
```

 Behind the scenes, JSX converts into JavaScript.

4. What are Props?

Props (properties) are used to **pass data from parent to child components**.

They are **read-only**, meaning child cannot modify them.

👉 Example:

```
<User name="Shreyas" />
```

● 5. What is State?

State is used to store **dynamic data inside a component**.

When state changes, React automatically **re-renders the UI**.

👉 Example:

Counter value changing on button click.

● 6. Props vs State?

Props

Passed from parent

Read-only

Used for communication

State

Managed inside component

Can be updated

Used for UI changes

● 7. What is Virtual DOM?

Virtual DOM is a **lightweight copy of the real DOM**.

When something changes:

1. React updates Virtual DOM
2. Compares with previous version (diffing)
3. Updates only changed parts in real DOM

👉 This improves performance.

● 8. What is SPA (Single Page Application)?

SPA loads a **single HTML page** and updates content dynamically without reloading.

👉 Benefits:

- Faster navigation
 - Better user experience
-

● 9. What is useState?

useState is a **hook used to manage state** in functional components.

```
const [count, setCount] = useState(0);
```

👉 count = value

👉 setCount = function to update value

● 10. What is useEffect?

useEffect is used to handle **side effects** like:

- API calls
- Timers
- Lifecycle events

```
useEffect(() => {  
  fetchData();  
}, []);
```

👉 [] means it runs only once (on mount)

● 11. When does useEffect run?

- No dependency → runs on every render
 - Empty array [] → runs once
 - With dependency [x] → runs when x changes
-

12. What is useRef?

useRef is used to:

- Access DOM elements directly
- Store values without re-render

👉 Example:

Focus an input field.

13. What is useMemo?

useMemo is used to **optimize performance** by avoiding unnecessary recalculations.

👉 Example:

If a heavy calculation depends on data, it runs only when data changes.

14. What is useCallback?

useCallback is used to **prevent function re-creation** on every render.

👉 Helps avoid unnecessary re-rendering of child components.

15. What is Conditional Rendering?

Rendering UI based on conditions.

```
{isLoggedIn ? <Home /> : <Login />}
```

16. What is List & Key in React?

When rendering lists, React needs a **unique key** to identify each element.

```
users.map(user => <li key={user.id}>{user.name}</li>)
```

👉 Helps React update efficiently.

● 17. What is Event Handling?

Handling user actions like click, input, submit.

```
<button onClick={handleClick}>Click</button>
```

● 18. What is Controlled Component?

Form elements controlled by React state.

👉 Example:

Input value comes from state.

● 19. What is Uncontrolled Component?

Form elements controlled by DOM (using refs).

👉 Less preferred.

● 20. What is React Router?

Used for navigation between pages in React apps without reloading.

👉 React Router

● 21. Difference: Link vs <a> tag?

- Link → no page reload
- <a> → reloads page

👉 Always use Link in React apps.

● 22. What is API Calling in React?

Usually done using useEffect.

```
useEffect(() => {  
  fetch("/api/users")  
}, []);
```

👉 Fetch data when component loads.

● 23. What is Lifting State Up?

Moving state to a **common parent component** so multiple child components can share it.

● 24. What is Context API?

Used to share data globally (without passing props manually at every level).

👉 Example:

User login data, theme.

● 25. What causes re-render in React?

- State change
- Props change

👉 React re-renders component when data changes.

🎯 What to Do Next

👉 First learn these 25 properly

👉 Practice speaking answers aloud

👉 Build small project

If you want:

👉 I'll give **remaining 25 (advanced with real interview answers)**

👉 Or take your **mock interview (React + AWS)**

Just tell me 👉

give remaining 25

Perfect 👍 — now let's complete the **remaining 25 (Advanced + Interview-Level)** with **clear explanations**, not short lines.

📘 Advanced React Interview Questions (26–50)

● 26. What is Reconciliation?

Reconciliation is the **process React uses to update the DOM efficiently**.

👉 Steps:

1. New Virtual DOM is created
2. Compared with old Virtual DOM (diffing)
3. Only changed parts are updated in real DOM

👉 This makes React fast.

● 27. What is Diffing Algorithm?

Diffing is the **comparison process** between old and new Virtual DOM.

👉 React checks:

- Which elements changed
- Which stayed same

👉 Based on keys and structure.

● 28. What is React.memo?

React.memo is used to **prevent unnecessary re-rendering of components**.

👉 If props don't change → component doesn't re-render

👉 Used for performance optimization.

● 29. What is `useLayoutEffect`?

Similar to `useEffect` but runs **before the browser paints the UI**.

👉 Difference:

- `useEffect` → after render
- `useLayoutEffect` → before paint

👉 Used when you need DOM measurements.

● 30. What is Prop Drilling?

Passing data through multiple layers of components even if intermediate components don't need it.

👉 Problem:

- Makes code messy

👉 Solution:

- Context API
-

● 31. What is Context API?

Used to **share data globally** across components.

👉 Avoids prop drilling

👉 Example:

- User authentication
 - Theme
-

● 32. What is Higher Order Component (HOC)?

A function that **takes a component and returns a new component**.

👉 Used for:

- Reusing logic (like authentication, logging)

● 33. What is Custom Hook?

A reusable function that uses React hooks.

👉 Example:

useFetchData()

👉 Helps reuse logic across components.

● 34. What is Lazy Loading in React?

Loading components **only when needed** instead of loading everything at once.

👉 Improves performance.

● 35. What is Suspense?

Used with lazy loading to **show fallback UI (like loading spinner)** while component loads.

● 36. What is Error Boundary?

A component that catches JavaScript errors in child components and shows fallback UI.

👉 Only works in class components.

● 37. What is Redux?

👉 Redux

Redux is a **state management library** used to manage global state in large applications.

👉 Flow:

Action → Reducer → Store → UI

● 38. When should you use Redux?

Use Redux when:

- App is large
- Many components share state
- Complex state management

👉 Not needed for small apps.

● 39. What is Strict Mode?

Strict Mode is a tool that helps identify:

- Unsafe code
- Deprecated features

👉 It runs extra checks in development.

● 40. What is Hydration?

Hydration is the process where React attaches event handlers to HTML rendered from server.

👉 Used in SSR apps.

● 41. What is Server-Side Rendering (SSR)?

Rendering React components on the **server instead of browser**.

👉 Benefits:

- Better SEO
 - Faster initial load
-

● 42. CSR vs SSR?

CSR

SSR

Render on browser Render on server

Slower first load Faster first load

Less SEO friendly SEO friendly

● 43. What causes performance issues in React?

- Too many re-renders
- Heavy calculations
- Large component tree

👉 Fix using:

- `useMemo`
 - `useCallback`
 - `React.memo`
-

● 44. How to optimize React performance?

- Use memoization
 - Split components
 - Lazy loading
 - Avoid unnecessary state
-

● 45. What is key role in performance?

Keys help React:

- Identify elements
- Avoid re-rendering entire list

👉 Always use unique keys.

● 46. What is Fragment?

Fragment (<> </>) allows grouping elements without adding extra DOM node.

● 47. What is default props?

Default values for props if not passed.

👉 Prevents undefined errors.

● 48. What is children prop?

Special prop used to pass content inside component.

<Card>Content</Card>

👉 Content is children.

● 49. What is folder structure in large apps?

Organized structure improves scalability:

components/

pages/

services/

hooks/

utils/

● 50. What are best practices in React?

- ✓ Use functional components
- ✓ Keep components small
- ✓ Use proper naming
- ✓ Handle errors
- ✓ Avoid unnecessary re-renders

3 Components in React

◆ Functional Components (Most Used)

```
function Hello() {  
  return <h1>Hello World</h1>;  
}
```

◆ Class Components (Old)

```
class Hello extends React.Component {  
  render() {  
    return <h1>Hello</h1>;  
  }  
}
```

👉 Use Functional Components (modern approach)

4 JSX (JavaScript XML)

- Syntax to write HTML inside JavaScript

```
const element = <h1>Hello</h1>;
```

Rules:

- Must return single parent element
- Use className instead of class
- Use {} to write JS inside JSX

```
const name = "Shreyas";  
<h1>Hello {name}</h1>
```

5 Props (Data Passing)

- Used to pass data from parent → child

```
function User(props) {  
  return <h1>{props.name}</h1>;  
}
```

```
<User name="Shreyas" />
```

👉 Props are **read-only**

6 State (useState Hook)

- Used to store dynamic data

```
import { useState } from "react";
```

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <>  
      <h1>{count}</h1>  
      <button onClick={() => setCount(count + 1)}>Click</button>  
    </>  
  );  
}
```

7 Event Handling

```
<button onClick={handleClick}>Click</button>
```

```
function handleClick() {  
  alert("Clicked");  
}
```

8 Conditional Rendering

```
{isLoggedIn ? <h1>Welcome</h1> : <h1>Login</h1>}
```

9 Lists & Keys

```
const users = ["A", "B", "C"];
```

```
users.map((user, index) => (  
  <li key={index}>{user}</li>  
));
```

👉 Keys help React identify elements

10 useEffect Hook (Very Important)

Used for:

- API calls
- Side effects

```
import { useEffect } from "react";
```

```
useEffect(() => {  
  console.log("Component mounted");  
}, []);
```

👉 [] → runs once

1 1 Forms (Basic)

```
function Form() {  
  const [name, setName] = useState("");  
  
  return (  
    <input  
      value={name}  
      onChange={(e) => setName(e.target.value)}  
    />  
  );  
}
```

1 2 API Calls

```
useEffect(() => {  
  fetch("https://api.example.com/users")  
    .then(res => res.json())  
    .then(data => console.log(data));  
}, []);
```

👉 Can also use axios

1 3 Routing

Using:

👉 React Router

```
<Route path="/home" element={<Home />} />
```

1 4 Folder Structure (Basic)

```
src/  
├── components/  
├── pages/  
├── App.js  
└── index.js
```

1 5 Virtual DOM

- React creates a **virtual copy of DOM**
 - Updates only changed parts
 - 👉 Improves performance
-

1 6 Controlled vs Uncontrolled Components

Controlled:

- React controls form data

Uncontrolled:

- DOM controls data (less used)
-

1 7 Important Interview Questions

◆ What is React?

→ JS library for UI

◆ Difference between State & Props?

- Props → read-only, parent to child
- State → mutable, inside component

◆ What is Virtual DOM?

→ Lightweight copy of real DOM

◆ What is useEffect?

→ Handles side effects (API, lifecycle)

◆ Why keys in list?

→ Identify elements uniquely

1 8 What You DON'T Need Now

- ✗ Redux
- ✗ Context API (advanced use)
- ✗ Class components deep dive